

Benchmarking FAISS Indexing Methods on the Deep1M Dataset:

A Comparative Study of Exact and Approximate Nearest Neighbour Search

Kha. Mo. Syeed Asif
ID: 2022-3-60-030
East West University

Abstract—Efficient nearest-neighbour (NN) search over million-scale vector databases is a fundamental challenge in information retrieval, recommendation systems, and computer vision. This paper presents a systematic empirical evaluation of five FAISS indexing strategies—FLATIP, IVFFLAT, IVFPQ, HNSW, and LSH—on the Deep1M dataset, a 96-dimensional angular-distance benchmark derived from deep image features. Experiments are conducted at two scales (100K and 1M vectors) and measured across four dimensions: build time, average query latency, Recall@10, and memory footprint. Results show that HNSW achieves the lowest query latency (≈ 0.38 ms at 1M scale) with strong recall (0.972), while IVFFlat delivers the best accuracy–speed tradeoff (recall 0.996 at 6.26 ms). IVFPQ achieves extreme memory compression (14 MB for 1M vectors) at the cost of significant recall loss. LSH underperforms on all metrics at scale. These findings provide actionable guidance for FAISS index selection in production systems.

Keywords: approximate nearest neighbour search, FAISS, HNSW, IVF, product quantisation, LSH, Deep1M, vector indexing

I. INTRODUCTION

The explosive growth of high-dimensional embeddings produced by deep neural networks has made scalable vector similarity search a critical infrastructure component. Given a query $\mathbf{q} \in \mathbb{R}^d$ and a database of N vectors, the goal is to retrieve the k most similar entries under a chosen metric. Brute-force exact search is $O(Nd)$ and becomes prohibitive for $N \geq 10^6$ in latency-sensitive settings.

Approximate Nearest Neighbour (ANN) methods trade bounded recall loss for dramatic latency reduction. The FAISS library [1] by Meta AI provides production-grade CPU and GPU implementations of multiple ANN families and is widely adopted in both industry and research.

Despite widespread use, practitioners face a non-trivial index selection problem. Build time, query latency, recall, and memory footprint trade off differently across methods and scale. This paper contributes: (1) a full empirical comparison of five FAISS indices at 100K and 1M scales; (2) recall–latency scatter analysis; (3) hyperparameter sensitivity curves for IVFFlat (n_{probe}) and HNSW (efSearch); and (4) practical recommendations backed by measured data.

II. DATASET DESCRIPTION

A. Deep-Image-96-Angular (Deep1M)

We use the *deep-image-96-angular* benchmark from ANN-Benchmarks [2], a 96-dimensional angular-distance dataset of deep CNN activations on natural images. Key properties are listed in Table I.

TABLE I
DATASET PROPERTIES

Property	Value
Full training vectors	9,990,000
Test query vectors	10,000
Dimensionality	96
Distance metric	Angular (cosine)
Ground truth per query	100 neighbours
Format / size	HDF5 / ≈ 3.58 GB

B. Subsets and Preprocessing

Two subsets are used: **100K** (first 100,000 training vectors) and **1M** (first 1,000,000). Queries are the first 500 test vectors. All vectors are ℓ_2 -normalised prior to indexing; under this normalisation inner product equals cosine similarity, enabling uniform use of METRIC_INNER_PRODUCT.

III. INDEXING METHODS

A. IndexFlatIP — Exact Baseline

Exhaustive inner-product scan over all N vectors. Recall@10 = 1.0 by construction; build is a memcopy ($O(Nd)$); query is $O(Nd)$; memory = $N \cdot d \cdot 4$ bytes.

B. IndexIVFFlat — Inverted File

k -means partitions the space into n_{list} Voronoi cells; only the n_{probe} nearest cells are scanned at query time [3]. Vectors are stored uncompressed.

C. IndexIVFPQ — Product Quantisation

Combines IVF with Product Quantisation [4]: each 96-dim vector is split into $M = 12$ sub-vectors of 8 dims, encoded with 8 bits (256 centroids), yielding 12 bytes per vector—a $32\times$ compression over `float32`.

D. IndexHNSWFlat — Graph Navigation

Builds a multi-layer proximity graph [5]; each node has $M = 32$ bidirectional links. `efConstruction` = 200 governs build quality; `efSearch` governs query exploration. Query complexity is $O(\log N)$.

E. IndexLSH — Locality-Sensitive Hashing

Projects vectors onto $n_{\text{bits}} = 256$ random hyperplanes to produce binary hash codes; retrieval uses Hamming distance. Memory = $n_{\text{bits}}/8$ bytes per vector; no query-time hyperparameter.

IV. EXPERIMENTAL SETUP

All experiments run on a single CPU node (Python 3.12, `faiss-cpu` 1.13.2, `numpy` 2.0.2, seed 42). Each index is built once; query latency is measured per query over 500 queries and averaged. No GPU acceleration is used. Recall@10 is defined as:

$$\text{Recall@10} = \frac{1}{Q} \sum_{q=1}^Q \frac{|\mathcal{R}_q \cap \mathcal{G}_q|}{10} \quad (1)$$

where $\mathcal{R}_q$ is the set of 10 retrieved neighbours and $\mathcal{G}_q$ is the exact ground truth. Index hyperparameters are listed in Table II.

TABLE II
INDEX HYPERPARAMETERS

Index	Parameter	Value
IVFFlat 100K	$n_{\text{list}}, n_{\text{probe}}$	256, 32
IVFFlat 1M	$n_{\text{list}}, n_{\text{probe}}$	1024, 64
IVFPQ 100K	$M, n_{\text{bits}}, n_{\text{probe}}$	12, 8, 32
IVFPQ 1M	$M, n_{\text{bits}}, n_{\text{probe}}$	12, 8, 64
HNSW	$M, \text{efConstr.}, \text{efSearch}$	32, 200, 64
LSH	n_{bits}	256

V. RESULTS AND ANALYSIS

A. Summary Tables

Tables III and IV report all metrics. Bold marks the best value per column.

TABLE III
BENCHMARK RESULTS — 100K VECTORS

Index	Build(s)	Lat.(ms)	R@10	Mem(MB)
FlatIP	0.039	6.194	1.000	38.4
IVFFlat	2.275	0.783	0.995	38.4
IVFPQ	6.992	0.343	0.423	1.3
HNSW	51.48	0.327	0.992	38.4
LSH	0.208	1.169	0.435	3.2

B. Build Time Comparison

FlatIP builds in under 0.5 s at 1M (raw memcopy). IVFFlat needs k -means training (25.9 s). IVFPQ is costlier (49.5 s) due to sub-quantiser fitting. HNSW dominates at 971 s, reflecting its $O(N \log N)$ graph construction. LSH builds in 3.6 s, the fastest approximate index.

TABLE IV
BENCHMARK RESULTS — 1M VECTORS

Index	Build(s)	Lat.(ms)	R@10	Mem(MB)
FlatIP	0.438	62.155	1.000	384.0
IVFFlat	25.93	6.262	0.996	384.0
IVFPQ	49.53	4.928	0.298	14.0
HNSW	971.5	0.384	0.972	384.0
LSH	3.557	6.124	0.339	32.0

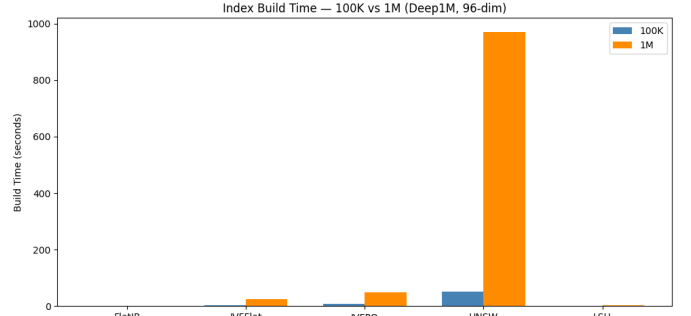


Fig. 1. Index build time (seconds) for 100K and 1M vectors. HNSW requires ≈ 971 s (≈ 16 min) at 1M due to $O(N \log N)$ graph construction; all other indices build within 50 s.

C. Query Latency

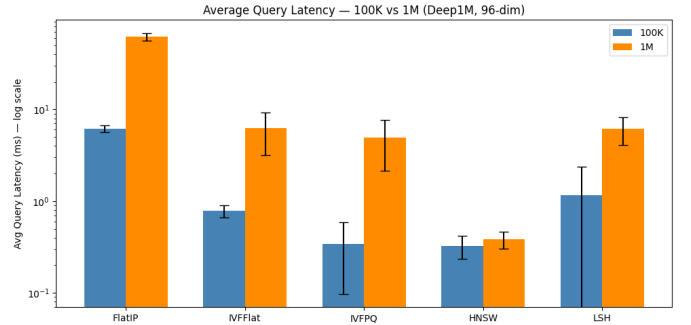


Fig. 2. Average per-query latency (ms, log scale) with standard deviation error bars. HNSW maintains sub-millisecond latency at both scales.

FlatIP scales linearly: 6.19 ms (100K) to 62.16 ms (1M). HNSW is nearly constant at 0.33–0.38 ms—a $163\times$ speedup over FlatIP at 1M—confirming its $O(\log N)$ complexity. IVFFlat grows from 0.78 to 6.26 ms ($8\times$), sub-linear relative to $10\times$ data growth. IVFPQ at 1M (4.93 ms) is slower than at 100K (0.34 ms) because doubling n_{probe} shifts the bottleneck back to IVF cell scanning.

D. Recall@10

FlatIP achieves perfect recall by construction. IVFFlat and HNSW both maintain >0.97 recall across both scales, confirming their suitability for accuracy-sensitive applications. IVFPQ recall drops from 0.42 (100K) to 0.30 (1M)—quantisation error compounds over a larger, more diverse database. LSH achieves 0.44 (100K) and 0.34 (1M) with no query-time tuning knob.

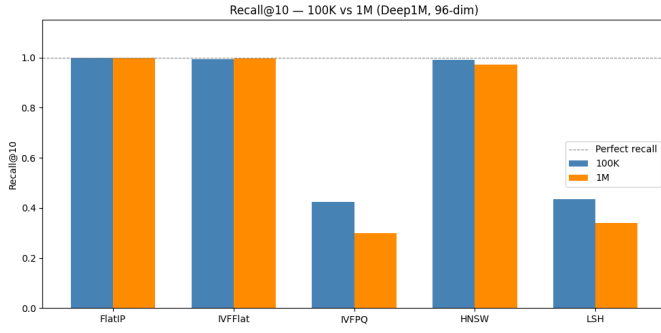


Fig. 3. Recall@10 across all five indices at 100K and 1M scales. The dashed line marks perfect recall. IVFPQ and LSH show substantial degradation.

E. Memory Footprint

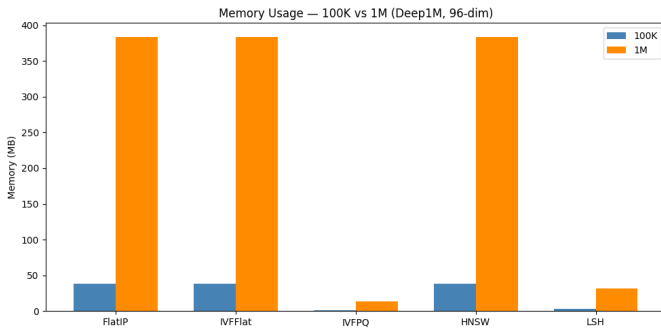


Fig. 4. Index memory consumption (MB). IVFPQ uses only 14 MB at 1M, achieving a $27\times$ compression over float32-based indices.

IVFPQ uses only 1.3 MB (100K) and 14 MB (1M) vs. 384 MB for float32-based indices—a $27\times$ compression. LSH uses 32 MB at 1M. This makes IVFPQ the only viable index for memory-constrained deployments, at the cost of recall.

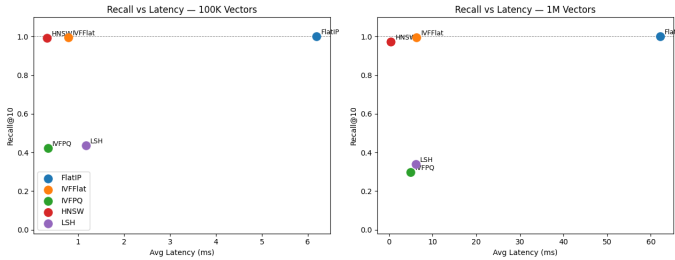


Fig. 5. Recall@10 vs. average query latency scatter plots for 100K (left) and 1M vectors (right). The upper-left corner is ideal (high recall, low latency). HNSW and IVFFlat are Pareto-dominant; IVFPQ and LSH are Pareto-dominated at both scales.

Figure 5 visualises the fundamental recall–latency tradeoff. HNSW and IVFFlat cluster in the upper-left (ideal) region at both scales: HNSW achieves the lowest latency while IVFFlat achieves near-perfect recall. IVFPQ and LSH are strictly dominated—the same or lower recall at equal or higher latency compared to IVFFlat. FlatIP sits in the upper-right (high recall, high latency) at 1M, confirming its unsuitability for interactive workloads.



Fig. 6. Scalability of build time (left), query latency (centre), and Recall@10 (right) from 100K to 1M vectors. HNSW build time dominates at scale; HNSW latency is nearly flat; IVFPQ recall degrades most.

Figure 6 shows how each metric evolves from 100K to 1M. The left panel confirms HNSW’s super-linear build cost; the centre panel highlights that only HNSW and FlatIP have predictable latency scaling—HNSW near-constant, FlatIP strictly linear. The right panel exposes a critical weakness of IVFPQ: recall degrades from 0.42 to 0.30, whereas IVFFlat and HNSW remain stable above 0.97.

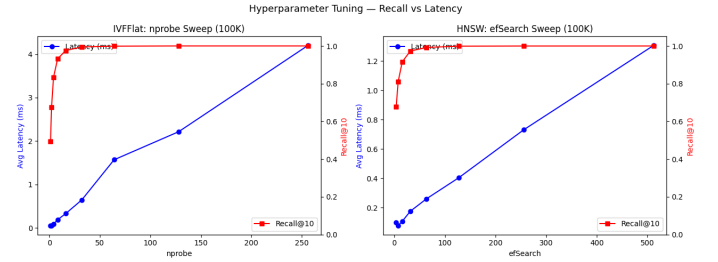


Fig. 7. Hyperparameter sweep curves for IVFFlat (nprobe, left) and HNSW (efSearch, right) on the 100K subset. Blue (left axis): average latency in ms. Red (right axis): Recall@10. Both methods show a clear elbow beyond which recall gains saturate while latency keeps rising.

Both tuning curves in Figure 7 exhibit a characteristic elbow. For IVFFlat, $nprobe=32$ delivers recall 0.995 at 0.65 ms; beyond this point latency grows faster than recall improves. For HNSW, $efSearch=64$ is the natural operating point (recall 0.992 at 0.26 ms). Both methods support smooth run-time adjustment of the recall–speed tradeoff without index rebuilding.



Fig. 8. Normalised performance heatmap (green = better) for 100K (left) and 1M vectors (right). Each metric is independently normalised to $[0, 1]$; lower build time, latency, and memory are inverted so green always means better.

The heatmap (Fig. 8) provides an at-a-glance multi-metric comparison. No single index dominates all four columns: FlatIP is best in recall but worst in latency at scale; HNSW leads on speed but is worst in build time; IVFPQ wins on memory but is worst in recall. IVFFlat achieves the most consistently green profile across build time, latency, and recall—confirming its status as the best all-round choice.

VI. DISCUSSION AND CONCLUSIONS

A. Key Findings

Four distinct operating regimes emerge from the data:

- 1) **FlatIP** — exact, zero tuning, zero recall loss. Suitable only at sub-100K scale or offline batch workloads where 62 ms per query is acceptable.
- 2) **IVFFlat** — best all-round choice. Recall 0.996 at 6.26 ms (1M); tunable via `nprobe`; moderate build cost (26s). The recommended default for latency-tolerant production systems.
- 3) **HNSW** — best for latency-critical workloads. Sub-millisecond queries at recall 0.972 (1M), tunable via `efSearch`. Pay the 16-min build cost once; query millions of times.
- 4) **IVFPQ** — the memory champion (14 MB at 1M). Recall ≈ 0.30 limits standalone use; combine with re-ranking (IndexIVFPQR) for practical deployment.

LSH offers no compelling tradeoff: recall 0.34 at 6.12 ms (1M) is strictly dominated by IVFFlat (recall 0.996 at the same latency).

B. Scalability

HNSW latency increases only 16% ($0.33 \rightarrow 0.38$ ms) from 100K to 1M, confirming $O(\log N)$ complexity. IVFFlat latency grows $8\times$ —sub-linear relative to $10\times$ data growth. IVFPQ recall degradation from 0.42 to 0.30 is a practical concern that must be addressed via re-ranking in production.

C. Recommendations

- Use **HNSW** when P_{99} query latency < 1 ms is required and build time can be amortised (e.g., static embedding indices).
- Use **IVFFlat** as the general-purpose default; tune `nprobe` to hit your recall SLA with minimal latency overhead.
- Use **IVFPQ** (+ re-ranking) when RAM is the binding constraint.
- Avoid **LSH** on FAISS for $d = 96$ datasets; it is Pareto-dominated by IVFFlat on every metric.

D. Limitations and Future Work

This study evaluates CPU throughput only. GPU-accelerated FlatIP and IVFFlat can yield $10\text{--}100\times$ throughput gains. Future work should evaluate: (i) batch query throughput (QPS); (ii) the effect of dimensionality; (iii) hybrid indices (HNSW+PQ, IndexIVFPQR); and (iv) recall at $k=100$ with out-of-distribution queries.

E. Conclusion

We benchmarked five FAISS indices on the Deep1M angular dataset at 100K and 1M scales across four performance dimensions. HNSW dominates on query latency; IVFFlat provides the best recall–speed balance; IVFPQ excels on memory efficiency. The heatmap, scalability, and scatter plots together confirm that no single index is universally optimal: the right choice depends on deployment constraints. Both IVFFlat

and HNSW are highly tunable, exposing well-defined recall–latency operating curves that practitioners can target without index rebuilding.

REFERENCES

- [1] J. Johnson, M. Douze, and H. Jégou, “Billion-scale similarity search with GPUs,” *IEEE Trans. Big Data*, vol. 7, no. 3, pp. 535–547, 2021.
- [2] M. Aumüller, E. Bernhardsson, and A. Faithfull, “ANN-Benchmarks: A benchmarking tool for approximate nearest neighbor algorithms,” *Inf. Syst.*, vol. 87, p. 101374, 2020.
- [3] H. Jégou, M. Douze, and C. Schmid, “Searching with quantization: Approximate nearest neighbor search using short codes and distance estimators,” *INRIA Tech. Rep.*, 2011.
- [4] H. Jégou, M. Douze, and C. Schmid, “Product quantization for nearest neighbor search,” *IEEE Trans. PAMI*, vol. 33, no. 1, pp. 117–128, 2011.
- [5] Y. A. Malkov and D. A. Yashunin, “Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs,” *IEEE Trans. PAMI*, vol. 42, no. 4, pp. 824–836, 2020.